Knowledge Representation & Extraction

Simple study notes for the project (based on Prof. Aldo Gangemi's slides)

How to read these notes: **yellow text** = the most important ideas. `grey code` = things you will actually type.

Blue boxes = tips for the final project.

Contents

(If the table of contents is empty, right-click it in Word and choose “Update Field”.)

0. The course and the exam in one page

The course is about one big question: **how do we turn data and text into knowledge that a computer can query and reason with?**

There are two halves:

- **Knowledge Representation (KR):** how to write knowledge down in a formal way. Tools: knowledge graphs, RDF, RDFS, OWL, SPARQL, ontology design patterns.
- **Knowledge Extraction (KE):** how to get knowledge out of messy sources (CSV files, databases, web pages, free text). Tools: scraping, linguistic patterns, NLP, machine learning, entity linking.

The exam = a group project

- **Groups of about 3 students (3 ± 1).**
- You pick a topic from the teachers' list of proposals.
- **You build an ontology + a knowledge graph about that topic**, using methods for **extraction**, **design** and **presentation** of knowledge.
- At the end you **present** the project.

What you should be able to do at the end

1. Understand the basics of KR, reasoning and extraction on the Semantic Web.
2. Use tools to reason and extract knowledge automatically.
3. **Analyse what a "customer" needs (requirements) and make a plan to build it.**

Playgrounds (places to practise SPARQL)

- DBpedia: https://dbpedia.org/sparql (general knowledge from Wikipedia)
- Wikidata: https://query.wikidata.org/
- ArCo (Italian cultural heritage): https://dati.beniculturali.it/sparql
- Framester (linguistic + factual knowledge), IMDB on QLever, Hugging Face, ChatGPT.

> **Project tip:** Most slide examples use **ArCo** and **DBpedia**. They are perfect for testing queries and for linking your own data to existing data.

1. Knowledge graphs and the Semantic Web

What is a knowledge graph (KG)?

There is no single official definition. The course uses this one (Hogan et al.):

A knowledge graph is a graph of data about the real world. Nodes are things (entities). Edges are relations between those things.

Example: [Umberto Eco] —wrote→ [The Name of the Rose].

Why graphs? (advantages)

- Easy and intuitive way to show complex relations (edges and paths).
- Query languages can do joins and also follow paths of any length.
- **You don't need a fixed schema at the start**: data can grow freely. (This is also dangerous: it can get messy. That is why we need ontologies.)
- Ontologies and rules let a computer **reason** about the data.
- You can do graph analytics (centrality, clustering) and machine learning on graphs.

A tiny bit of history

- The term exists since 1972.
- Tim Berners-Lee introduced Linked Data in 2006.
- It became famous in 2012 with the **Google Knowledge Graph**.

The Semantic Web

The normal web links **documents** (HTML pages) with links that have no meaning. A computer only sees "page A links to page B".

The Semantic Web gives meaning to things and links, so computers can understand them. Links get a type (e.g. "is author of") and things can be people, places, events — not only web pages.



Main W3C standards (the "language family"):

Standard	What it does (simple words)
RDF	The basic data model: everything is a triple (subject – predicate – object).
RDFS	A small vocabulary to describe classes and properties (types, hierarchies, domain/range).
OWL	A rich logic language for ontologies. Allows more reasoning.
SPARQL	The query language for RDF (like SQL for graphs).
RIF / SWRL	Languages for rules (IF … THEN …).
JSON-LD, Turtle, RDF/XML	Different ways (syntaxes) to write RDF.

Two key messages from lecture 2

1. **Separate knowledge from its presentation.** The data is one thing; how it looks on a web page is another.

2. **Aim for interoperability.** Example: Rome and Milan publish hotel data with different column names (INSEGNA vs insegna, TOTALE_CAMERE vs Numero camere…). This is **semantic heterogeneity**. A shared ontology solves it.

URIs / IRIs: names for things

A **URI** (or IRI, the international version) is a unique ID for a thing on the web, e.g. `http://dbpedia.org/resource/Miles_Davis`.

- Rule: given a URI, a machine should get RDF, a human should get an HTML page (this is called **content negotiation**).
- Do not confuse the URI of a **web page about** Miles Davis with the URI of **Miles Davis the person**.

Berners-Lee's 4 Linked Data principles

1. **Use URIs as names for things.** Bir linki okuneg: `http://dbpedia.org/resource/Miles_Davis`.
2. Use HTTP URIs so people can look them up.
3. When someone looks up a URI, give useful information using standards (RDF, SPARQL).
4. **Include links to other URIs** so people can discover more things.

4. başka linkleri de içeriyor

FAIR data

- **F**indable – permanent URIs (e.g. w3id.org).j
- **A**ccessible – open protocols (HTTP, SPARQL).
- **I**nteroperable – standard models (RDF, OWL).
- **R**eusable – open licence (e.g. CC-BY 4.0).

Where do we get Semantic Web data? Four solutions

Solution	How	Example tools
1. Import	Convert a relational database to RDF	R2RML (W3C standard)
2. Extract	Scraping + NLP from web pages and text	TagMe, DBpedia Spotlight, FRED
3. Export	Embed RDF inside web pages	RDFa, microformats
4. Reuse	Use existing Linked Open Data	DBpedia, Wikidata, LOD cloud

> **Project tip:** In your project you will probably mix **extract** (from CSV or text) and **reuse** (link your entities to DBpedia/Wikidata with `owl:sameAs`).

2. RDF – the basic data model

The triple

Everything in RDF is a triple: Subject – Predicate – Object. A set of triples = an RDF graph.

Subject	Predicate	Object
Umberto Eco	is author of	“The name of the rose”
dbr:Miles_Davis	dbo:recordLabel	dbr:Columbia_Records

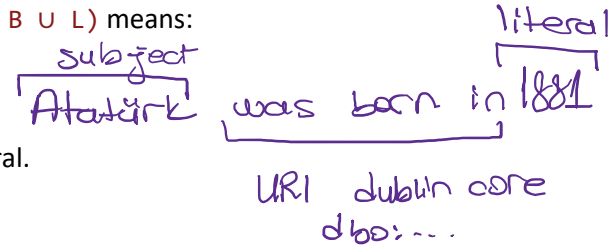
Three kinds of nodes

- **IRI / URI (resource)**: a named thing, e.g. `dbr:Miles_Davis`.
- **Literal**: a value like a string, number or date, e.g. `"Miles"`, `"1926-05-26"^^xsd:date`. It can have a language tag: `"Gibson's Neuromancer"@en`.
- **Blank node**: a thing with no global name, only a local ID, e.g. `_:id1` or `[ ... ]`. You can replace it with a real IRI (“skolemization”).

Where can each kind go? (the formal rule)

`t = (s, p, o) ∈ (U ∪ B) × U × (U ∪ B ∪ L)` means:

- **Subject**: an IRI or a blank node.
- **Predicate**: **always an IRI.**
- **Object**: an IRI, a blank node, or a literal.
- **A literal can never be a subject.**



Turtle syntax (the one you will write)

```
@prefix rdf:     <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs:    <http://www.w3.org/2000/01/rdf-schema#> .
@prefix dbpedia: <http://dbpedia.org/resource/> .
@prefix myont:   <http://www.myont.org#> .

myont:Neuromancer rdf:type myont:BedsideBook .
myont:Neuromancer myont:author dbpedia:William_Gibson .
myont:Neuromancer rdfs:label "Gibson's Neuromancer"@en .
myont:Neuromancer myont:pages "258"^^xsd:int .
_:id1 myont:includes myont:Neuromancer .
```

- Every triple ends with a **dot (.)**
- `;` = same subject, new predicate. `,` = same subject and predicate, new object.
- `a` is a shortcut for `rdf:type`.
- `@prefix` saves you from writing full URIs.

Types of things in a graph (logic level)

- **Individual**: a single thing (Neuromancer, Miles Davis).
- **Class**: a group/type of things (Book, Person).
- **Object property**: links two things (author).
- **Datatype property**: links a thing to a value (pages → 258).
- **Annotation property**: for humans, no logic (rdfs:label, rdfs:comment).

Why we need more than “just triples”: the CSV example

A CSV row `Miles,Davis,102,1926-05-26` converted with a tool like csv2rdf gives you one **blank node per row** with columns as properties. **But what is that blank node? A person? A fact? Just a row?** The tool does not know. You must decide (e.g. give it the ID `:Miles_Davis` and the type `Person`). **This is exactly why we need ontologies and design patterns.**

3. SPARQL – querying the graph

SPARQL is the standard query language for RDF. **It works by pattern matching: you write a pattern with variables (`?x`), and SPARQL finds all parts of the graph that fit.**

Key concepts

- **Triple pattern**: a triple with variables, e.g. `?agent a-cd:isAuthorOf ?work`.
- **Basic Graph Pattern (BGP)**: several triple patterns together (logical AND).
- **Join**: when two triple patterns share the same variable. That shared variable is the join variable.
- **Result**: a set of **bindings** (values for the variables).
- **Graph traversal**: you can chain patterns to walk across the graph: event → cultural entity → author → label.

The 4 query types

Query	Returns	Use it for
SELECT	A table of values	Normal questions (most common)
CONSTRUCT	A new RDF graph	**Creating/transforming triples** (very useful for projects!)
ASK	true / false	"Is there any … ?"
DESCRIBE	RDF about a resource	"Tell me everything about X"

Basic SELECT

```
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX cis:  <http://dati.beniculturali.it/cis/>

SELECT ?subject ?label
WHERE {
  ?subject rdfs:label ?label .
  ?subject rdf:type cis:GeographicalFeature .
}
```

The keywords you need

Keyword	What it does	Example
OPTIONAL	**Add info if it exists, but don't drop the result if missing**	`OPTIONAL { ?x a-cd:depiction ?img }`
UNION	Either pattern A or pattern B	`{ ?x :author :A } UNION { ?x :author :B }`
FILTER	Keep only results that pass a test	`FILTER(REGEX(?name, "michelangelo", "i"))`
DISTINCT	Remove duplicates	`SELECT DISTINCT ?x`
ORDER BY	Sort (ASC default, DESC)	`ORDER BY DESC(?name)`
LIMIT	Max number of results	`LIMIT 10`
OFFSET	Skip the first N results (paging)	`OFFSET 10`
GRAPH	Which named graph the triples come from	`GRAPH ?g { ... }`

Notes: `"i"` in REGEX = case-insensitive. LIMIT/OFFSET only make sense together with ORDER BY. Useful FILTER operators: `! && ||`, `= != > <`, `IN`, `isURI`, `isLiteral`, `bound`, `str`, `lang`, `regex`.

CONSTRUCT – build new triples (very useful!)

Example: rename `rdfs:label` to `l0:name` for all works by Andrea del Castagno in Florence events:

```
CONSTRUCT {
  ?cp a-cd:hasAuthor ?author ; l0:name ?title .
}
WHERE {
  ?event clv:hasSpatialCoverage ?place ;
         cis:involvesCulturalEntity ?cp .
  ?place rdfs:label "Firenze" .
  ?cp a-cd:hasAuthor ?author ; rdfs:label ?title .
  ?author rdfs:label ?authorName
  FILTER(REGEX(?authorName, "andrea del castagno", "i"))
}
```

CONSTRUCT can also work as a rule: "if a drug has a component with a function, then the drug is a commodity for that function".

ASK and DESCRIBE

```
ASK { ?cp a-cd:hasAuthor ?a . ?a rdfs:label ?n
      FILTER(REGEX(?n, "michelangelo", "i")) }

DESCRIBE agent:50f36ef3d01e9761a295106ebbd39de3
```

A real problem: data cleaning (Michelangelo example)

In ArCo the same artist appears with many name variants: "Michelangelo Buonarroti (attribuito)", "Buonarroti, Michelangelo - 1475 - 1564", "Michelangelo Buonarroti (scuola)"… Some are the real person, some mean "attributed to", "his school", "his style", "a copy". **Entity deduplication is hard**: you need names + dates + works, not only string similarity. ArCo chose **high precision** (few mistakes) and lost some **recall** (some duplicates remain).

> **Project tip:** Write your competency questions as SPARQL queries. If a query returns the right answer, your ontology passes the test. This is how you test an ontology.

Bir entity isimini verirken name+date+works gibi ayrıntılarda vermeliyiz. Veriyi bulurken en uygun olanı getiriyor.

4. RDFS – a first vocabulary for meaning

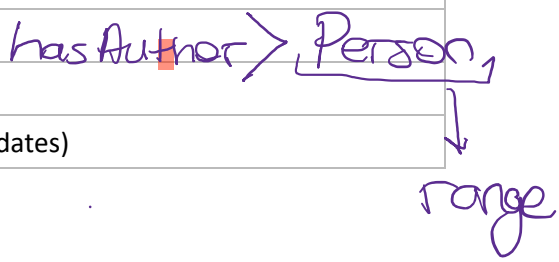
What is an ontology?

In philosophy: the study of what exists. In computer science: **a formal model of the main concepts and relations of a domain.** (Gruber: a set of "representational primitives" – classes, properties, relations – with their meaning and constraints.)

- **TBox** (terminological box) = the schema: classes and properties. E.g. "Book is a subclass of Work".
- **ABox** (assertional box) = the data: facts about individuals. E.g. "Neuromancer is a Book".

RDFS main elements

Term	Meaning in simple words
rdfs:Resource	The class of everything
rdf:type	"x is a member of class C"
rdfs:Class	A set of things
rdf:Property	A relation (a set of pairs)
rdfs:subClassOf	Every member of A is also a member of B
rdfs:subPropertyOf	If x P1 y then also x P2 y
rdfs:domain	The class of the subject of a property
rdfs:range	The class of the object of a property
rdfs:label	A human-readable name
rdfs:Literal	The class of values (strings, numbers, dates)



RDFS inference rules (entailments) – the important ones

IF you have…	THEN the computer infers…
P rdfs:domain C + x P y	x rdf:type C
P rdfs:range C + x P y	y rdf:type C
A subClassOf B + z type A	z type B
A subClassOf B + B subClassOf C	A subClassOf C (transitive)
P1 subPropertyOf P2 + x P1 y	x P2 y

The big trap: domain and range do NOT check errors

RDF vs OWL farkı

If `hasAuthor` has range `Person` and you write `:sinopia :hasAuthor :giotto_event`, RDFS does **not** say "error!". **It simply infers that giotto_event is a Person.** RDF is very "liberal". RDFS gives you inferences, not constraints.

bu bir event

To catch such mistakes you need OWL (e.g. disjoint classes: Person and Event can't overlap → the reasoner says "inconsistent").

OWL ile hata tespit ediyoruz!

Other RDF(S) features (just know they exist)

- **Containers**: rdf:Bag (unordered), rdf:Seq (ordered), rdf:Alt (choose one). Members: rdf:_1, rdf:_2…
- **Collections / Lists**: rdf:first, rdf:rest, rdf:nil.
- **Reification**: a statement about a statement (“Wikipedia says that McCarthy invented Lisp”) using rdf:Statement, rdf:subject, rdf:predicate, rdf:object.

> **Project tip:** The homework on the slides (Music vocabulary: band, members, roles, artist, music project, time) is a great warm-up for the project. Draw it with **Graffoo** (yEd or draw.io).

5. OWL – the Web Ontology Language

OWL is a logic-based language for ontologies. **It is richer than RDFS, so a reasoner can (1) check if your knowledge is consistent and (2) make hidden knowledge explicit.**

RDFS vs OWL

- OWL includes everything from RDFS.
- It adds: set operations (and/or/not), `owl:sameAs`, `owl:equivalentClass`, restrictions (some/only/cardinality), property characteristics (transitive, symmetric…), property chains, keys.
- OWL also tells you what you **cannot** do (e.g. you can't freely mix class and individual).
- More power = slower reasoning. OWL has **profiles** to choose how expressive you want to be.
- Reuse other ontologies with `owl:imports`.

Basic notions

- **Axiom**: a basic statement ("every man is mortal"). An ontology = a set of axioms.
- **Entailment**: A ⊨ a means "whenever all statements in A are true, a is true too".
- **Consistency**: there is at least one possible world where all statements are true together. If not → inconsistent.
- **Entities**: individuals, classes, object properties, datatype properties, annotation properties.
- **Expressions**: combinations, e.g. `Son ≡ Person ⊓ Male`.

Core OWL in Turtle + Manchester syntax

Protégé shows Manchester syntax. Turtle is what you write in files.

What	Manchester syntax (Protégé)
Instance	`Individual: x  Types: CulturalProperty`
Subclass	`Class: PhotographicHeritage SubClassOf: MovableCulturalProperty`
Equivalent class	`Class: Person EquivalentTo: Human`
Disjoint	`DisjointClasses: Person, CulturalProperty`
Domain / range	`ObjectProperty: hasAuthor Domain: CulturalProperty Range: Person`
Same / different	`SameAs: zeri:andrea-del-Castagno` / `DifferentFrom: …`
Data value	`Facts: dateOfBirth "1419"^^xsd:gYear`

Boolean class constructors

- **and** (intersection): `TangibleCulturalProperty ≡ CulturalProperty and PhysicalObject`
- **or** (union): `CulturalProperty ≡ Tangible… or Intangible…`
- **not** (complement): `IntangibleCulturalProperty ≡ CulturalProperty and not PhysicalObject`

Restrictions: some vs only (very important!)

Manchester	DL	Meaning
`P some B`	∃P.B	**At least one** P-value is a B. (existential)

Manchester	DL	Meaning
`P only B`	∀P.B	**All** P-values (if any) are B. (universal)
`P value x`	∃P.{x}	Has P with the specific individual x
`P min/max/exactly n B`	≥n, ≤n, =n	Cardinality

Example: `CulturalPropertyRecord SubClassOf: describes some CulturalEntity` = every record describes at least one cultural entity.

SubClassOf vs EquivalentTo (necessary vs sufficient)

- **SubClassOf** = **necessary condition**: "If you are an OmnivoreAnimal, then you eat animals and plants." (The reasoner will NOT classify new things into this class.)
- **GCI (General Class Inclusion)** = sufficient condition: "If an animal eats animals and plants, then it is an OmnivoreAnimal."
- **EquivalentTo** = **definition (necessary + sufficient)**. The reasoner CAN automatically classify individuals into this class.
- Real fuzziness ("when is a heap no longer a heap?") needs fuzzy OWL.

Property characteristics

Characteristic	Meaning	Example
Functional	Only one value (two values → they are the same)	hasMother, hasAge
Inverse functional	Only one subject per value	DNAof
Symmetric	a P b ⇒ b P a	hasSpouse, hasBrother
Asymmetric	a P b ⇒ never b P a	hasMother
Transitive	a P b and b P c ⇒ a P c	hasPart, isLocatedIn
Reflexive	Everything is P to itself	sameAs
Irreflexive	Nothing is P to itself	differentFrom

Other useful OWL 2 tricks

- **Property chains (rules inside OWL)**: `brotherOf o motherOf SubPropertyOf uncleOf` → Mustafa brother of Saida, Saida mother of Rashid ⇒ Mustafa uncle of Rashid.
- **Keys (hasKey)**: identify individuals by values. `Transplantation hasKey [donor, recipient, organ]` – two transplantations with the same three values are the same.
- **Punning**: the same name used as a class and an individual (Supervisor as a role, a class and a relation). The reasoner keeps the meanings separate.
- **Defined datatypes**: `Age ≡ xsd:integer[>0] and xsd:integer[<150]`; `Minor ≡ Human and hasAge Age[<18]`.
- **Rules in other forms**: SPARQL CONSTRUCT, SWRL (`hasDrugComponent(?x,?y) ^ hasFunction(?y,?z) -> commodityFor(?x,?z)`), RIF.

> **Project tip:** In Protégé: create classes, properties, individuals → run the **reasoner** (HermiT/Pellet). Try putting a person into a class disjoint with Person and watch the inconsistency error.

error yakalamak

6. Ontology Design Patterns (ODPs)

An Ontology Design Pattern is a reusable, successful solution to a recurring modelling problem. Idea comes from architecture (Christopher Alexander) and software engineering (Gang of Four). Patterns are small problem–solution pairs.

Examples of problems: "How do I represent a relation with more than 2 arguments in OWL?", "How do I represent parts and wholes?"

ODPs and requirements

- **Competency Questions (CQs)**: questions the ontology must answer. "How many people flew from Rome to Berlin in 2019?"
- **Negative CQs**: things that must NOT be possible. "How many people flew twice on the same flight?"
- **Stories**: concrete text. "Chad Smith was the drummer of the Red Hot Chili Peppers when they recorded Stadium Arcadium (Sep 2004 – Dec 2005)."
- **Scenarios**: abstract version. "A person plays a role in a band during an album recording in a time interval."
- Requirements are **matched against patterns** and tested with **unit tests** (usually SPARQL queries).
- Patterns have alternatives with pros and cons. There are also **anti-patterns** (common mistakes).

Catalogue of patterns: **http://www.ontologydesignpatterns.org** (e.g. taskrole.owl, transition.owl, observation.owl, invoice.owl…).

Types of patterns (NeOn project)

- **Content ODPs** – domain modelling (roles, situations, parts…). **These are the ones you will use most.**
- **Logical ODPs** – logic structure, e.g. n-ary relation trick, rule macros.
- **Reasoning ODPs** – classification, inheritance, materialization…
- Others: Structural, Architectural, Lexico-syntactic, Presentation (naming, annotation), Correspondence (reengineering, alignment).

Pattern layers (from logic to text)

Logical pattern → Generic knowledge pattern → Specific knowledge pattern → Data → Text. Example:

```
?x a owl:Class ; rdfs:subClassOf ?y . ?y a owl:Restriction .        # logic
:Inflammation rdfs:subClassOf [:localisedIn some :BodyPart] .         # generic
:Colitis rdfs:subClassOf [:localisedIn some :Colon] .                 # specific
:colitis-j :localisedIn :colon-j .                                    # data
"John has got colitis"                                                # text
```

Anti-patterns (mistakes to avoid)

Wrong ✗	Right ✓	Why
`:City subClassOf :Country`	`:City subClassOf [partOf some Country]`	A city is a PART of a country, not a KIND of country
`:City subClassOf :Geography`	`:City skos:broader :Geography`	Topic classification ≠ subclass
`:Dead or :Alive`	`:Dead disjointWith :Alive`	"or" in language often means disjoint
`:Pen and :Paper`	`:Pen or :Paper` / a Collection	"and" in language often means union

Wrong ✗	Right ✓	Why
`:BankBehavior subClassOf :Crisis`	`:BankBehavior :isCauseOf :Crisis`	Causality is not subsumption
`:dog(word) rdf:type :Dog`	`:dog(word) :expresses :Dog`	A word is not a dog!
two rdfs:domain values	domain = union of the two	Two domains = intersection (usually not what you want)
`John_Coltrane ∈ Miles_Davis_Group`	`John_Coltrane memberOf Miles_Davis_Group`	**A collection is not a class**

The n-ary relation problem (the most important pattern)

OWL properties are binary (2 arguments). But real facts often have more. "Chad Smith played drums in RHCP for the album Stadium Arcadium from 2004 to 2005" has **5** arguments: person, role, band, album, time.

Solution: Situation pattern (also called Frame Occurrence, n-ary reification).

1. **Reify**: make the whole fact an individual of a new class (`PlaySituation`).
2. **Specialise** a general pattern: `PlaySituation subClassOf sit:Situation`.
3. **Binary projections**: one property per argument (personPlaying, playsRole, inBand, forAlbum, recordingTime).
4. **Identification constraint**: `PlaySituation hasKey [all 5 properties]`.

```
:PlaySituation rdfs:subClassOf sit:Situation .
:ps1 a :PlaySituation ;
     :personPlaying :Chad_Smith ;
     :playsRole     :Drummer ;
     :inBand        :Red_Hot_Chili_Peppers ;
     :forAlbum      :Stadium_Arcadium ;
     :recordingTime :sep2004_dec2005 .
```

Time indexing: the "mother of all patterns" (Hayes)

How to add time (or space, role, opinion…) to a fact R(a,b)?

- **Sentence indexing**: put the time on the whole triple (quads / named graphs).
- ****Relation indexing**: reified n-ary relation** (the Situation pattern above).
- **Individual indexing**: time slices of things (4D, "John-in-2020").
- **Predicate indexing**: singleton property (a unique property per fact).
- **Name nesting**: ad hoc names like `wasPresidentIn2006` (simple but not scalable).

Role patterns – 6 ways to say "John Coltrane was the sax player"

Pattern	How it looks	Good for
Role as class	`John_Coltrane a :SaxPlayer`	Simple, quick
Role as individual	`John_Coltrane :hasRole :sax_player`	Talking about the role itself
Role punning	`SaxPlayer ≡ hasRole value sax_player`	Both at once
Role as property	`John_Coltrane :player :Sax`	Very simple, binary
Role as situation	`:s1 r1 :John ; r2 :Player ; r3 :Sax ; r4 :date`	**Time, place, context (n-ary)**
Role as trope/qua-entity	`John hasTrope :JohnAsSaxPlayer`	A "slice" of a person

Parts and wholes – different relations, different logic

Relation	Transitive?	Example
Whole – Part	Yes	country / city
System – Component	No	car / engine
Entity – Constituent	Yes	statue / clay
Collection – Member	No	exhibition / painting
Set – Element	No	redheaded / Irish redheaded
Aggregate – Fragment	Yes	heap / stuff

In OWL 2: transitivity can't be combined with asymmetry. Trick ("reflexive reduction"): `properPartOf subPropertyOf partOf`; `partOf` transitive + reflexive; `componentOf subPropertyOf properPartOf`, componentOf asymmetric.

Other useful content patterns (from the ODP portal)

- **Tasks & Roles / Task execution / Control flow** – workflows, recipes, instructions.
- **Transition** – an event changes the state of an object at a time.
- **Trajectory** – moving objects, fixes (place + time), segments.
- **Contracts / Invoices** – parties, line items, prices.
- **Observation** – an observation of something, with parameters and a date.
- ****Descriptions & Situations (D&S)****: a Description (norm, plan, diagnosis, opinion) defines Concepts; a Situation **satisfies** a Description. Used for laws, medical diagnoses, perspectives.
- **Information realization**: Dante's Comedy (information) vs. a printed copy, an ebook, a reading (realizations).

Foundational ontologies

- **DOLCE / DOLCE-Ultralite (DUL)**: top-level categories. **Endurant ≈ object**, **Perdurant ≈ event**, Quality, Region (abstract, physical, temporal).
- Aligning DBpedia to DOLCE found 3.65M inconsistencies (only 0.7% visible without the alignment) → foundational ontologies help find errors.
- Cognitive roots: frames (Fillmore, Minsky), scripts (Schank), schemata. **FrameNet** and **VerbNet** give language-based patterns (e.g. the Cure frame: Healer, Patient, Medication, Affliction…).

> **Project tip:** For almost every project: events with participants + time + place → use the **Situation / n-ary pattern**. People with jobs/positions → a **role pattern**. Groups → the **collection pattern** (memberOf).

7. How to build an ontology: the XD method

eXtreme Design (XD) is the method used in the course (and for the big ArCo project). It is agile, like extreme/pair programming.

XD workflow (simplified)

1. **Get the project context** with the customer: domain? task?
2. **Collect requirement stories** (texts, experts, datasets).
3. Pick one story.
4. **Turn the story into Competency Questions (CQs).**
5. Pick one CQ.
6. **Match the CQ to a design pattern** (General Use Case).
7. Select and reuse the pattern(s).
8. Integrate them into your module.
9. **Test and fix** (SPARQL unit tests, reasoner).
10. Repeat until all CQs and stories are done; release the module; integrate modules.

Experiments show: patterns + XD **improve ontology quality** (coverage, fewer errors, better design).

Worked example: from a news sentence to an ontology

Story (New York Times): **"Leaders in Ferguson, Mo., said Darren Wilson, the police officer who submitted a letter of resignation after a grand jury declined to indict him, would receive no pension, severance package or other benefits."**

Step A – Colour the text

- **Events**: said, submitted, declined, indict, receive.
- **Classes**: leader, police officer, letter, resignation, grand jury, pension, benefit.
- **Individuals**: Ferguson, Missouri, Darren Wilson (+ anonymous ones: "a letter", "a grand jury").
- **Relations**: in, of, after. **Logical terms**: no, or, other.

Step B – Write CQs, match a pattern

CQ	Problem	Pattern
Where are the leaders? What is Ferguson part of?	Location of a group	individual + partOf (+ collection)
Who said what?	Relation between two events	**eventReificationWithRoles**
What is the role of Darren Wilson?	Person's role	roleAsClass or roleReification
Who submitted what, when? Who indicted whom?	Event with >2 arguments; "who/him"	event reification + coreference
Who receives what? (no…, or…, other…)	Negation, "other"	negatedEvent + superclass

Step C – The model (part)

```
:say_1     a :Say ;     :agent :leaders_1 ; :topic :receive_1 .
:receive_1 a :Receive ; :agent :Darren_Wilson ; :isNegated "true" ;
           :patient :benefit ; :hasModality :necessary .
:submit_1  a :Submit ;  :agent :Darren_Wilson ; :patient :letter_1 ;
           :after :decline_1 .
```

```
:decline_1 a :Decline ; :agent :jury_1 ; :topic :indict_1 .
:jury_1    a :GrandJury .
:Darren_Wilson a :Person ; :hasRole :PoliceOfficer .
:Ferguson :partOf :Missouri .
:Pension rdfs:subClassOf :Benefit .
```

Notice: **events become individuals** (say_1, receive_1) with roles like `agent`, `patient`, `topic`. This is the event-reification pattern (Neo-Davidsonian style).

Step D – Link to the web (entity linking)

```
:Ferguson      owl:sameAs dbpedia:Ferguson,_Missouri .
:Darren_Wilson owl:sameAs dbpedia:Darren_Wilson .
:GrandJury     owl:equivalentClass dbpedia:Grand_jury .
:Receive       owl:equivalentClass verbnet:Receive_13052000 .
```

Step E – Make it grow

Use SPARQL CONSTRUCT on DBpedia to import facts about the linked entities into your graph.

The ArCo case (a real pattern-based project)

- Italian cultural heritage KG: 7 modules, ~5,000 axioms, 169M triples, ~20,000 owl:sameAs links.
- Requirements collected as **stories** (Google Forms), translated into **CQs**, continuous feedback (GitHub issues, mailing list).
- Pipeline: requirements → modules (pattern-based, modular) → data production (XML→RDF) → **testing (CQ verification, inference verification, error provocation)** → refactoring → release.
- Documentation with **LODE** (human-readable docs), published on GitHub, Zenodo, LOV; CC-BY licence.

> **Project tip:** Copy the ArCo approach for your project: stories → CQs → patterns → Turtle/OWL → data → SPARQL tests → documentation.

8. Knowledge Extraction

==Knowledge extraction = getting structured knowledge (RDF/OWL) from unstructured or semi-structured sources.== The course mixes rule-based methods (scraping, linguistic patterns) and statistical methods (machine learning).

Sources and how to extract from them

Source	Method / tool
CSV / tables	csv2rdf (mapping file: column → property)
Relational DB	R2RML
Wikipedia infoboxes	DBpedia extractor
XML	XML-to-RDF converters (used in ArCo)
Free text	NLP: NER, relation extraction, entity linking, FRED

Main NLP tasks (know the names)

- **NER (Named Entity Recognition)**: find names of people, places, organizations in text.
- **Entity linking** ("wikification"): connect the name to a public ID (Wikipedia/DBpedia). Tools: TagMe, Babelfy, DBpedia Spotlight.
- **Word Sense Disambiguation (WSD)**: which meaning of a word (link to WordNet/BabelNet).
- **Relation extraction**: find relations between entities ("X was born in Y").
- **Dependency parsing**: grammar structure of a sentence (CoreNLP, spaCy, Stanza).
- **Coreference resolution**: find that "he", "him", "who" refer to Darren Wilson.

Open Information Extraction vs Open Knowledge Extraction

- **Open IE** (e.g. ReVerb, AllenAI OpenIE): finds simple text triples like (he, arrived in, Sarajevo). No real semantics, no web IDs.
- ****Open Knowledge Extraction (OKE)****: goes further. Produces a real knowledge graph: entities and relations **linked to public web identities** (DBpedia, WordNet…), with logic and patterns, plus annotations on the text.

FRED – the course's main extraction tool

FRED (http://wit.istc.cnr.it/stlab-tools/fred/demo/) reads a sentence and outputs an RDF/OWL knowledge graph. It does:

- Events with semantic roles (Agent, Theme… from VerbNet), negation, modality, tense.
- Entity linking (DBpedia), sense linking (WordNet, FrameNet), types and taxonomy.
- Alignment to DOLCE (e.g. Play ⊑ dul:Event).

Example: "John Coltrane played with Miles Davis in Kind of Blue" →

```
fred:play_1 a fred:Play ;
    vn:Actor1 fred:John_coltrane ;
    vn:Actor2 fred:Miles_davis .
fred:Play rdfs:subClassOf dul:Event .
fred:John_coltrane owl:sameAs dbpedia:John_Coltrane .
```

Semiotic drifting: after linking to DBpedia, you can query DBpedia (e.g. DESCRIBE Kind_of_Blue) and expand your graph with related knowledge.

Why extraction is hard (4 classes of problems)

1. **Small NLP errors cause big errors** (POS tagging: “shares” as noun or verb; coordination; titles; plural coreference).
2. **Non-literal meaning in the text**: metonymy (“I ate a plate of lentils”), metaphor (“freeway of love”), fictive motion (“the road runs along the coast”).
3. **Hidden meaning (in absentia)**: implicatures, common sense, physical context (“pass me the glass over there”).
4. **Higher-order phenomena**: opinions, norms, arguments, narratives, text types (“A cat is on the mat” vs “A cat is a mammal”).

Adjectives are tricky

- “Sicilian surgeon” ⇒ a surgeon (and Sicilian). **Extensional** pattern.
- “Skilful surgeon” ⇏ skilful arsonist. Skill is relative to the role.
- “Alleged surgeon” ⇏ surely a surgeon. **Modal** pattern.
- “Fake surgeon” ⇒ NOT a surgeon. **Privative** pattern.

Beyond facts: opinions and sentiment (Sentilo)

From “Bhatkal’s father: I’m glad he has been arrested”, Sentilo finds the **opinion holder** (father), the **topic** (arrest), the **sentiment**, and the **impact** on the roles (negative for the arrested person).

Take-home message

We need several types of grounding: physical/cultural/social context, common-sense knowledge, multi-layer representations (facts vs opinions vs interpretations), and **hybrid neuro-symbolic systems** (knowledge graphs + embeddings / LLMs).

> **Project tip:** Run some sentences from your project texts through **FRED**: it gives you a first draft of classes, events and DBpedia links. Then clean it by hand using patterns.

9. Tools you can use

Ontology design and checking

- **Protégé**: the main OWL editor + reasoner.
- **Graffoo** (yEd / draw.io): draw RDFS/OWL diagrams.
- **WebVOWL**: visualise an OWL ontology.
- **TopBraid Composer Free**: RDF + SPARQL editor.
- **Turtle validator**, **RDF converter** (change syntax).
- **OOPS!** (OntOlogy Pitfall Scanner): finds common modelling mistakes.
- **LODE**: creates HTML documentation of your ontology.
- XD Tools, ComodIDE / OPLaTab (Protégé plugins for patterns).

Storing and querying (triplestores)

- **GraphDB**: store, query, reason, visualise.
- **Apache Jena Fuseki**: simple SPARQL server, good for small datasets.
- **Blazegraph**: high-performance, good for publishing Linked Data.

Extraction

- Structured data: **csv2rdf**, **R2RML**, **DBpedia extractor**.
- Text: **FRED**, **TagMe** and **Babelfy** (entity linking), **CoreNLP** (parsing), **OpenIE** (AllenAI), **NELL** (fact learning).
- Similarity / vectors: **word2vec**, **ESAlib**, **WordNet::Similarity**, Embedding Projector.

Visualisation and distant reading

- **RelFinder** (find paths between entities in DBpedia), **Gapminder**, **Google n-gram viewer**, **BookNLP** (annotate whole books).

10. Project checklist (step by step)

Use this as your plan. It follows the XD method from the slides.

1. **Choose the topic and the goal.** What is the domain? What should the KG be used for?
2. **Collect sources**: CSV files, websites, texts, existing datasets (DBpedia, Wikidata, ArCo).
3. **Write 3–5 stories and turn them into 10–20 Competency Questions.**
4. **Look for existing ontologies and patterns** (ontologydesignpatterns.org, schema.org, FOAF, DOLCE-UL, ArCo, CIDOC-CRM…). **Reuse before inventing.**
5. **Design the ontology** in Protégé: classes, properties, domain/range, restrictions, disjointness. Use patterns (Situation for n-ary facts, roles, collections, parts). Draw it with Graffoo.
6. **Avoid the anti-patterns** (partOf ≠ subClassOf, collection ≠ class, etc.).
7. **Extract / convert the data to RDF** (csv2rdf, SPARQL CONSTRUCT, Python + rdflib, FRED for text).
8. **Link your entities** to DBpedia/Wikidata with `owl:sameAs`; link classes with `owl:equivalentClass` / `rdfs:subClassOf`.
9. **Load into a triplestore** (GraphDB or Fuseki).
10. **Test**: write each CQ as a SPARQL query and check the answers; run the reasoner (no inconsistencies); check with OOPS!.
11. **Document** (LODE, a README, diagrams) and publish (GitHub).
12. **Prepare the presentation**: problem → requirements (CQs) → patterns used → data and extraction → example queries and results → limits and future work.

11. Mini glossary

Term	Simple meaning
ABox / TBox	Data (facts about individuals) / Schema (classes and properties)
Axiom	One basic statement in an ontology
Blank node	A node with no global name
BGP	Basic Graph Pattern: a group of triple patterns in SPARQL
Competency Question	A question the ontology must be able to answer
Consistency	No contradictions in the ontology
Entailment	Something that logically follows from what you said
Entity linking	Connecting a name in text to a public ID (e.g. DBpedia)
IRI / URI	Unique web identifier for a thing
Knowledge graph	Graph of entities (nodes) and relations (edges)
Linked Open Data	RDF data on the web, linked to other datasets, open licence
Literal	A value: string, number, date
Named graph	A set of triples with its own IRI
n-ary relation	A relation with more than 2 arguments (needs reification in OWL)
ODP	Ontology Design Pattern: reusable modelling solution
Ontology	Formal model of the concepts and relations of a domain
Reasoner	Program that checks consistency and infers new facts (HermiT, Pellet)
Reification	Turning a relation/fact into an individual so you can describe it
Triple	Subject – Predicate – Object
Triplestore	A database for RDF (GraphDB, Fuseki, Blazegraph)
Turtle	The most readable way to write RDF
XD	eXtreme Design: agile, pattern-based ontology method